discuss whether the same three regimes (too slow / good / diverging) show up here too, or whether real tabular data behaves differently than the toy sine problem.

Step 4: Experiment C — regularization (dropout)

This directly depends on resolving the dropout question from Step 1. You'll take your **largest** architecture from Experiment A (`(256, 128, 64)`) and run it twice — once with `dropout=0.0` (no dropout) and once with `dropout=0.3` — then compare train vs validation accuracy for both. This is testing whether dropout helps close the train/validation gap you noticed in your last plot (a real overfitting-reduction technique).

Before writing any code, the two things worth resolving first:

1. Does your `FeedForwardNet` class need to be rewritten to handle a *variable* number of hidden layers (for Experiment A) and an optional dropout parameter (for Experiment C)?
2. What exactly goes into the returned `history` dictionary from `run_experiment`?

Want to tackle the `FeedForwardNet` redesign first (making it flexible for any number of hidden layers + optional dropout), since that seems to be the prerequisite blocking all three experiments? Or would you rather start with the `run_experiment` function skeleton assuming the class is already flexible?

explain each of the steps you used to get the codes, what they are doing and why they are placed there, also with the intellicode in my vs code suggested some for me of which some are slighlty different from yours so why the difference and what does each of them mean for example running_loss += loss.item() * xb.size(0) from you and

```
        running_train_loss += loss.item()
 preds = torch.argmax(logits, dim=1) from you and  predicted =
torch.max(logits.data, 1)
total += xb.size(0) from you and  total_train += yb.size(0)

with this ones while I understand that some things where actually
computed and store in different varibales and used later the
method of calculating those things differed from the two
suggestions I had with the other one there were some additional
parameters which were added and I didn know where they were
coming from so exaplain your codes first what each one do and
then also clarify the confusion or differencies in the two
solutions
    epoch_train_loss = r  g_loss / total
    epoch_train_acc = correct / total
```

```
solutions
    epoch_train_loss = running_loss / total
    epoch_train_acc = correct / total
    train_losses.append(epoch_train_loss)
    train_accs.append(epoch_train_acc)
and     train_losses.append(running_train_loss /
len(train_loader))
    train_accuracy.append(100. * correct_train / total_train)
  if epoch_val_acc > best_val_acc:
        best_val_acc = epoch_val_acc and  if val_accuracy[-1] >
best_val_accuracy:
        best_val_accuracy = val_accuracy[-1]
```

Show less

22:52

Dissected code implementations and compared alternative approaches systematically

Let's go through this in two passes — first explaining what my code does line by line, then directly comparing it to the alternate version t… lear up the confusion.

Part 1: Explaining my code, piece by piece

Part 1: Explaining my code, piece by piece

`criterion = nn.CrossEntropyLoss()` — this creates your loss function object. It measures how wrong your model's predictions are compared to the true labels, specifically for multi-class classification.

`optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)` — this is the automated version of what you did *by hand* in Part 1.1 (`w -= lr * dw`). `model.parameters()` tells the optimizer which tensors it's allowed to update (all your `nn.Linear` weights and biases). Adam is a more sophisticated update rule than plain gradient descent — it adapts each parameter's effective learning rate individually.

The epoch loop (`for epoch in range(n_epochs)`) — everything inside runs once per full pass through your dataset.

`model.train()` — puts the model in "training mode." Doesn't do much for your current architecture, but it's the standard convention.

The inner batch loop (`for xb, yb in train_loader`) — this pulls one batch of 32 rows at a time from your DataLoader.

- `optimizer.zero_grad()` — PyTorch *acc lates* gradients by default rather than replacing them. If you didn't clear them each batch, gradients from previous batches

- `optimizer.zero_grad()` — PyTorch *accumulates* gradients by default rather than replacing them. If you didn't clear them each batch, gradients from previous batches would keep adding up incorrectly. This resets them to zero before computing new ones.
- `logits = model(xb)` — runs your forward pass (calls your `forward()` method automatically).
- `loss = criterion(logits, yb)` — computes how wrong this batch's predictions are.
- `loss.backward()` — this is PyTorch automatically computing every gradient (`dW1, db1, dW2, db2`, etc.) — the exact same chain-rule process you did manually in Part 1.2, just automated across your whole network at once.
- `optimizer.step()` — applies the update rule (like your manual `w -= lr*dw`) to every parameter, using the gradients just computed.

Now, my specific loss/accuracy tracking — this is where your question lives, so let's slow down:

```python
running_loss += loss.item() * xb.size(0)
```

`loss` is the average loss across this one batch (that's just how `CrossEntropyLoss` works

`loss` is the **average** loss across this one batch (that's just how `CrossEntropyLoss` works by default — it reduces to a mean). `xb.size(0)` is how many rows are in this specific batch (usually 32, but possibly fewer for the last batch if your dataset doesn't divide evenly). Multiplying "average loss" × "number of rows" converts it back into a **total** loss for this batch. I accumulate these totals across all batches, so that at the end of the epoch I can divide by the **true total row count** to get a correctly weighted overall average — this matters if your last batch is smaller than 32, since a plain average-of-averages would slightly misweight it.

python

```
preds = torch.argmax(logits, dim=1)
correct += (preds == yb).sum().item()
total += xb.size(0)
```

`logits` has shape `(batch_size, 7)` — 7 raw scores per row. `argmax(dim=1)` finds the *index* of the highest score per row, which corresponds to your model's predicted class. Comparing `preds == yb` gives a True/False per row; `.sum()` counts how many were correct in this batch. `total` accumulates the al row count seen so far (again, to handle any uneven final batch correctly).

```python
epoch_train_loss = running_loss / total
epoch_train_acc = correct / total
```

Divide the accumulated totals by the accumulated row count to get this epoch's true average loss and accuracy (as a fraction between 0 and 1).

Part 2: Comparing to your IntelliCode-suggested version

Here's the key insight: **both versions are valid — they're just using slightly different (and slightly less precise) simplifying assumptions.** Let's go term by term.

`running_train_loss += loss.item()` vs my `loss.item() * xb.size(0)`

The suggested version just adds the raw average batch loss directly, without weighting by batch size.

```python
train_losses.append(running_train_loss / len(train_loader))
```

Then at the end, it divides by `len(train_loader)` — which is the **number of batches**, not the number of rows. This computes an "average of batch averages." **Question to think about:** is this exactly mathematically equal to the true overall average, if every batch has exactly the same size (32)? Yes, actually — when all batches are equal-sized, averaging the batch-averages gives the same result as my row-weighted approach. **Where they'd differ:** only if your dataset size isn't perfectly divisible by 32, making your last batch smaller — in that case, the suggested version very slightly overweights that smaller batch's influence, while mine weights strictly by actual row count. For most practical purposes, this difference is tiny and won't meaningfully change your results.

`predicted = torch.max(logits.data, 1)` vs my `torch.argmax(logits, dim=1)`

These do essentially the same job through a different route. `torch.max(tensor, dim)` returns **two things** — the actual maximum *values* AND their *indices* — as a tuple. So `torch.max(logits.data, 1)` alone would give you a tuple; the suggested code was likely written as `_, predicted = torch.max(logits.data, 1)` (using `_` to discard the values and keep only the indices), or you may have only pasted part of it. `torch.argmax(logits, dim=1)` more directly gives you *just* the indices, which is all you actually need here. `.data` is an older, mostly outdated way of accessing a tensor's raw values without gradient tracking — modern PyTorch code typically ju es the tensor directly, or wraps things in `torch.no_grad()` instead (which you're already doing correctly).

`total_train += yb.size(0)` vs my `total += xb.size(0)`

Functionally identical — `xb` and `yb` always have the same number of rows in a given batch (since they're paired), so counting rows from either one gives you the same number either way. Pure style choice, not a real difference.

`train_accuracy.append(100. * correct_train / total_train)` vs my `train_accs.append(epoch_train_acc)` (where `epoch_train_acc = correct/total`)

This is just a **scale difference**, not a logic difference. The suggested version multiplies by 100 to express accuracy as a percentage (e.g., `87.5`), while mine leaves it as a fraction (e.g., `0.875`). Both represent the exact same information — just check which your lab expects when you report/plot it, and be consistent with your axis labels (e.g., label the y-axis "Accuracy (%)" if you use the ×100 version).

`if val_accuracy[-1] > best_val_accuracy:` vs my `if epoch_val_acc > best_val_acc:`

Also functionally identical. `val_accuracy[-1]` grabs the most recently appended value from the list (Python's negative indexing for "last item"), which is exactly the same value I'm holding directly in `epoch_val_acc` before pending it. Different route to the same number.

Linking Google Colab to GitHub repository

Free plan · Upgrade

Share

python

```python
# Loss function and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
# Adam chosen over plain SGD because it adapts the learning rate per-parameter,
# which tends to converge faster and more reliably on tabular data like this,
# without needing as much manual tuning of a fixed lr (as you saw with your
# three-learning-rate experiment in Part 1.3).

# History tracking
train_losses, val_losses = [], []
train_accs, val_accs = [], []
best_val_acc = 0.0

n_epochs = 50

for epoch in range(n_epochs):
    # ---- Training phase ----
    model.train()
    running_loss = 0.0
```

Write a message...

Sonnet 5 Medium

Linking Google Colab to GitHub repository

Free plan · Upgrade

Share

```
for xb, yb in train_loader:
    xb, yb = xb.to(DEVICE), yb.to(DEVICE)

    optimizer.zero_grad()
    logits = model(xb)
    loss = criterion(logits, yb)
    loss.backward()
    optimizer.step()

    running_loss += loss.item() * xb.size(0)
    preds = torch.argmax(logits, dim=1)
    correct += (preds == yb).sum().item()
    total += xb.size(0)

epoch_train_loss = running_loss / total
epoch_train_acc = correct / total
train_losses.append(epoch_train_loss)
train_accs.append(epoch_train_acc)

# ---- Validation phase ----
model.eval()
```

Write a message...

Sonnet 5 Medium

```
model.eval()
val_running_loss = 0.0
val_correct = 0
val_total = 0

with torch.no_grad():
    for xb, yb in val_loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        logits = model(xb)
        loss = criterion(logits, yb)

        val_running_loss += loss.item() * xb.size(0)
        preds = torch.argmax(logits, dim=1)
        val_correct += (preds == yb).sum().item()
        val_total += xb.size(0)

epoch_val_loss = val_running_loss / val_total
epoch_val_acc = val_correct / val_total
val_losses.append(epoch_val_loss)
val_accs.append(epoch_val_acc)
```

This is a bigger task than previous TODOs — it's asking you to build a reusable experiment function, then run three separate comparisons through it. Let's break down what's actually being asked before writing anything.

The big picture

You already have a full working training loop (Part 2.3) sitting in a notebook cell. This TODO wants you to turn that loop into a **function** — similar to what you did with `train_toy()` back in Part 1.3 — so you can call it repeatedly with different settings without copy-pasting the whole loop three times.

Step 1: Wrapping into `run_experiment(...)`

Think back to how you built `train_toy(lr, steps=3000)` — same idea, but now with more parameters since you're testing more things:

- `hidden_sizes=(64, 32)` — controls your `FeedForwardNet`'s architecture (this already matches your class's constructor argument)
- `lr` — controls the optimizer's learning rate
- `batch_size` — controls your DataLoader's batch size
- `epochs` — controls how many times you
- `dropout=0.0` — a **new** concept not yet in your model. Think about this: your current

Write a message...

Sonnet 5 Medium

Question to think through: what should `run_experiment` actually *do* internally? Walk through the pieces it needs to recreate every time it's called:

1. Build a fresh model using the given `hidden_sizes` (and `dropout` if applicable)
2. Build fresh DataLoaders using the given `batch_size` (since batch size is now variable, unlike before)
3. Set up `criterion` and `optimizer` using the given `lr`
4. Run the training loop for the given number of `epochs`
5. Return a `history` dictionary — **question:** what should this dictionary contain? Look at what your three experiments actually need to plot afterward (validation accuracy curves, validation loss curves, train vs validation accuracy) — that tells you exactly which lists need to be in this returned dictionary (e.g., `train_losses`, `val_losses`, `train_accs`, `val_accs`).

Step 2: Experiment A — capacity

This tests whether a bigger network (more neurons/layers) performs better. You'll call `run_experiment()` three times, changing on `idden_sizes`, keeping everything else at defaults:

This tests whether a bigger network (more neurons/layers) performs better. You'll call `run_experiment()` three times, changing only `hidden_sizes`, keeping everything else at defaults:

- `hidden_sizes=(8,)` — tiny, just one hidden layer with 8 neurons
- `hidden_sizes=(64, 32)` — your current default
- `hidden_sizes=(256, 128, 64)` — large, three hidden layers

Question: does your `FeedForwardNet` class currently support an *arbitrary number* of hidden layers, or is it hardcoded for exactly two (`fc1`, `fc2`, `fc3`)? Look back at your class — it has fixed layers `self.fc1`, `self.fc2`, `self.fc3`. Would this handle `hidden_sizes=(8,)` (only one hidden layer) or `hidden_sizes=(256, 128, 64)` (three hidden layers) correctly? This is a significant design question to resolve before running Experiment A at all.

Step 3: Experiment B — learning rate

Same three-value comparison pattern from Part 1.3, but now on your real dataset instead of the toy sine problem. You'll call `run_experiment()` three times, varying only `lr`. The prompt specifically asks you to **compare with t 1.3** — meaning your write-up should discuss whether the same three regimes (too slow / good / diverging) show up here too, or